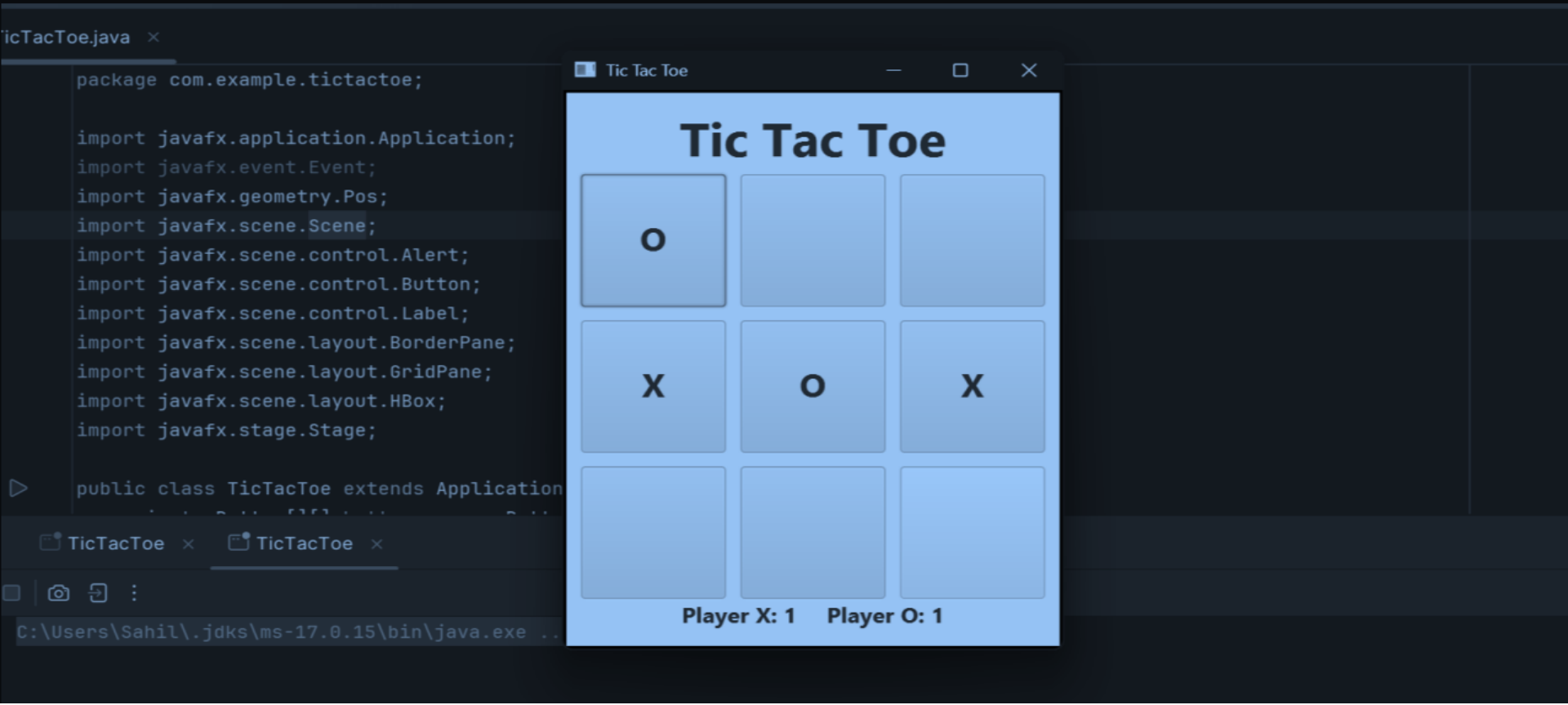


Java Programming

CPT111 – Lecture 12

Erick Purwanto

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lecture 12

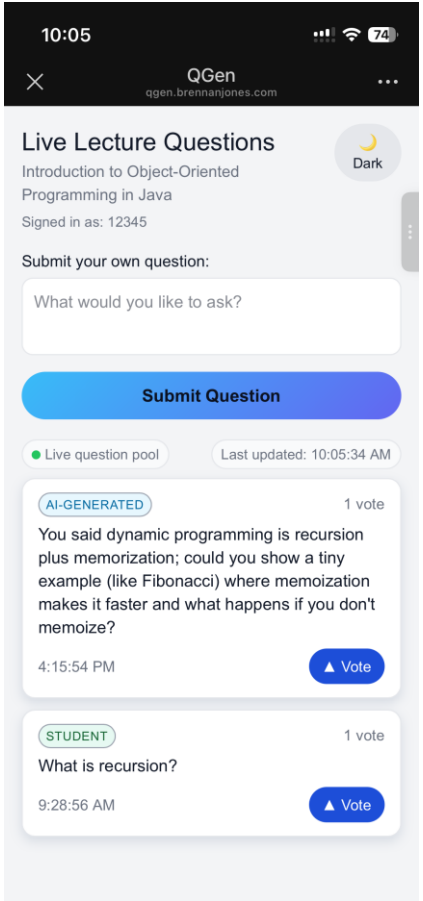
GUI with JavaFX

Welcome!

- Welcome to Lecture 12 !
- Last week, we have learned about Recursion

- In this lecture we are going to learn about
  - GUI with JavaFX
    - drawing shapes and creating graphical user interface
    - practicing OOP with JavaFX

Interactive Q&A with QGen



- In this lecture, we're going to try something different!
- We will have **Q&A during the lecture** – but we're going to do it with an app
  - Please login to qgen.brennanjones.com
    - Using your **student ID**
  - Every ~15 minutes there will be Q&A
    - You can anonymously **submit** your own questions
    - You can **vote** on questions
    - Some questions will be **AI-generated**
    - The instructor will answer the **top-voted questions**

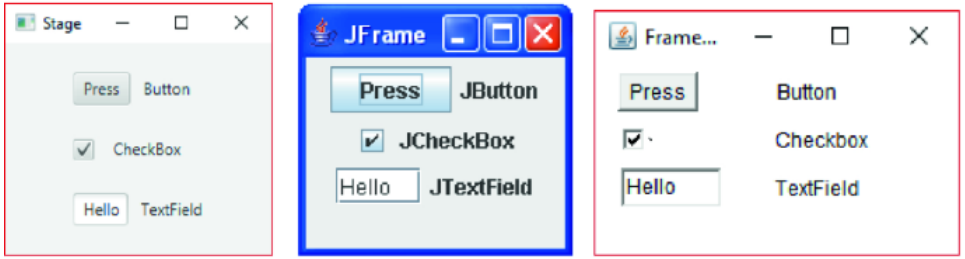
GUI with JavaFX

- **JavaFX** is a **GUI** toolkit for Java (GUI = Graphical User Interface)
  - up until now, we build TUI (Text-based User Interface)
  - JavaFX makes it easier to create desktop applications and games in Java
  - Platform-independent, can run in Windows, Linux, Mac, iOS, Android, Raspberry Pi

- Sources:
  - Introduction to Java Programming and Data Structure, 12th Edition, Y. Daniel Liang, **Chapter 14 – Java FX Basics** for the main reading
    - please read for full codes and explanations
  - **Chapter 15-16** for self-study (CW3), not tested in Final Exam

JavaFX vs Swing/AWT

- Previously, Java GUI classes are Abstract Windows Toolkit (AWT) and Swing
  - Depreciated, no longer updated
- JavaFX is the **new** framework that **replaces** them since Java 8
  - For **CW3**, if you choose GUI (optional), you must use JavaFX
  - Submissions with other platforms such as Swing/AWT are not accepted



JavaFX Swing AWT

JavaFX vs Swing

- Major differences:

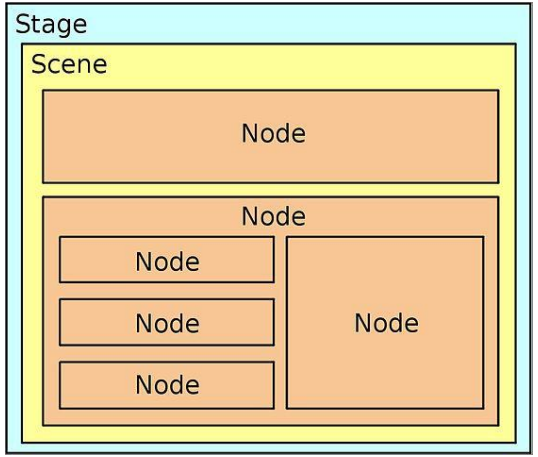
Feature	JavaFX	Swing
Property Bindings	JavaFX properties support binding, meaning you can listen for changes in their values.	Swing properties do not directly support binding.
Declarative Layout	JavaFX has support for declarative layout via FXML.	Swing has no built-in support for declarative layout
Styling	JavaFX supports CSS based and code based styling.	Swing only supports code based styling.
WebView	JavaFX has a WebView that can render modern web pages	Swing has no WebView.
Graphics	JavaFX uses vector based graphics.	Swing uses pixel based graphics.
3D Graphics	JavaFX has built-in support for 3D graphics.	Swing requires the Java 3D API for 3D graphics.
Concurrency API	JavaFX has a built-in concurrency API.	Swing has no built-in concurrency API.
Age of toolkit	JavaFX is newer.	Swing is older.
Included in Java SDK	JavaFX is not included from Java 11 and forward.	Swing is still included.

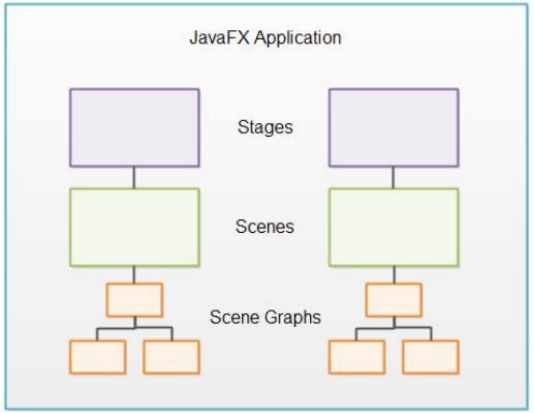
JavaFX Features

Core	Layout	Basic Controls		Container Controls	Web	Charts	Other Concepts
Stage Scene Node Properties FXML CSS	Region Pane HBox VBox FlowPane TilePane GridPane Group StackPane AnchorPane BorderPane	Label Button MenuButton SplitMenuButton ButtonBar ToggleButton RadioButton CheckBox ChoiceBox ComboBox ListView TextField PasswordField TextArea ImageView DatePicker ColorPicker	Slider Tooltip Hyperlink ProgressBar ProgressIndicator MenuBar ContextMenu Separator TableView TreeView TreeTableView HTMLEditor Pagination FileChooser DirectoryChooser Spinner	Accordion TitledPane TabPane SplitPane ScrollPane	WebView WebEngine	PieChart BarChart StackedBarChart ScatterChart LineChart AreaChart StackedAreaChart BubbleChart	Fonts TextFormatter Dialogs Color 2D Shapes 3D Shapes Effects Transformations Animation Canvas PixelBuffer Drag and Drop Audio Video Print API JavaFX concurrency

JavaFX Advantages

- **Simple** and excellent application of **OOP** with different layers:
  - **Stage**: top level container to be displayed
    A JavaFX application contains one or more stages which corresponds to windows
  - **Scene**: Scene objects are placed in a Stage object
    Each stage has a scene attached to it
  - **Node**: Node objects are placed in a Scene object
    - Node objects are visual components such as buttons, images, panes, and shapes
    - Node can contain other nodes
    - This nested structure is also called **Scene Graphs**

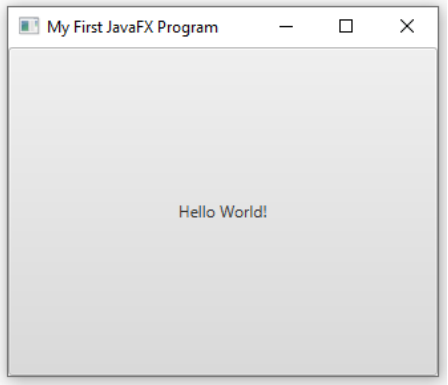


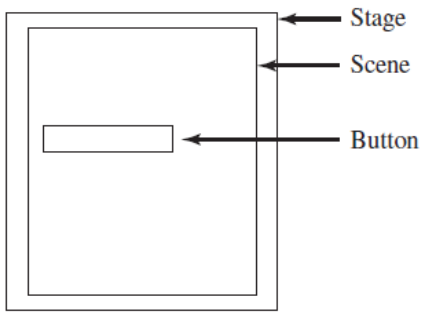


Hello GUI World!

- Our first JavaFX Program:

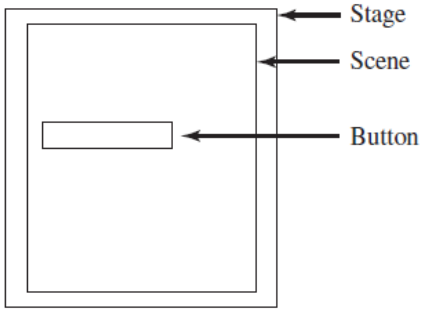
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;

public class JavaFXTesting extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button helloButton = new Button("Hello World!");
        Scene scene = new Scene(helloButton, 200, 250);
        primaryStage.setTitle("My First JavaFX Program");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```





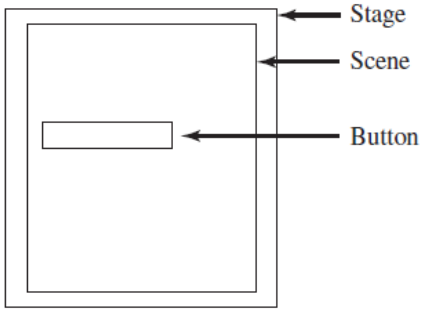
Stage

- The **Stage** is the outer frame of a JavaFX application
  - it typically corresponds to a window
  - a JavaFX app can have multiple windows open
  - each stage is represented by a Stage object inside a JavaFX application
  - a Stage object called **primary stage** is automatically created by JVM when the application is launched
  - a JavaFX application can create additional Stage objects if it needs additional windows open
    - for instance, dialog windows, wizards



Scene

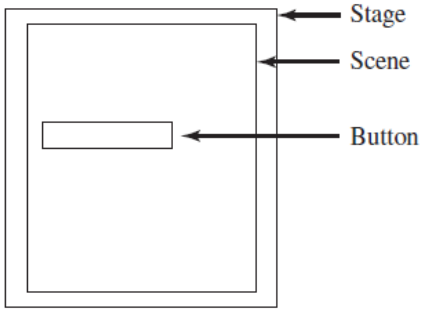
- The main class overrides **start** method defined in **Application**
  - start method **places** node objects such as a **Button** object in a **Scene** object
  - and **displays** Scene objects in a **Stage** object



- A stage can only show **one scene at a time**, but it is possible to exchange the scene at runtime (e.g., a game may have multiple screens)
  - during the life time of a JavaFX application, a Stage object in JavaFX can show multiple scenes (one at a time, all must be created)
- Scene objects are created by **Scene(node, width, height)**
  - the constructor specifies width, height and places **node** in the scene

Nodes

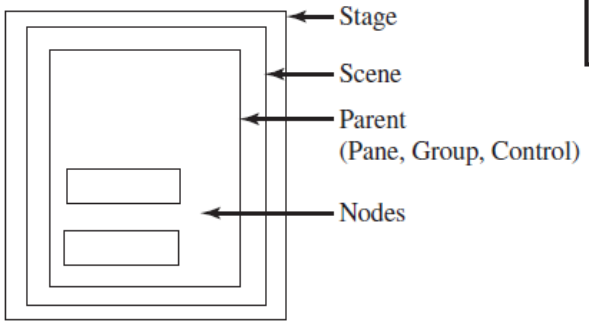
- All components attached to the scene graph are called **nodes**
- All nodes are subclasses of a JavaFX class called javafx.scene.Node

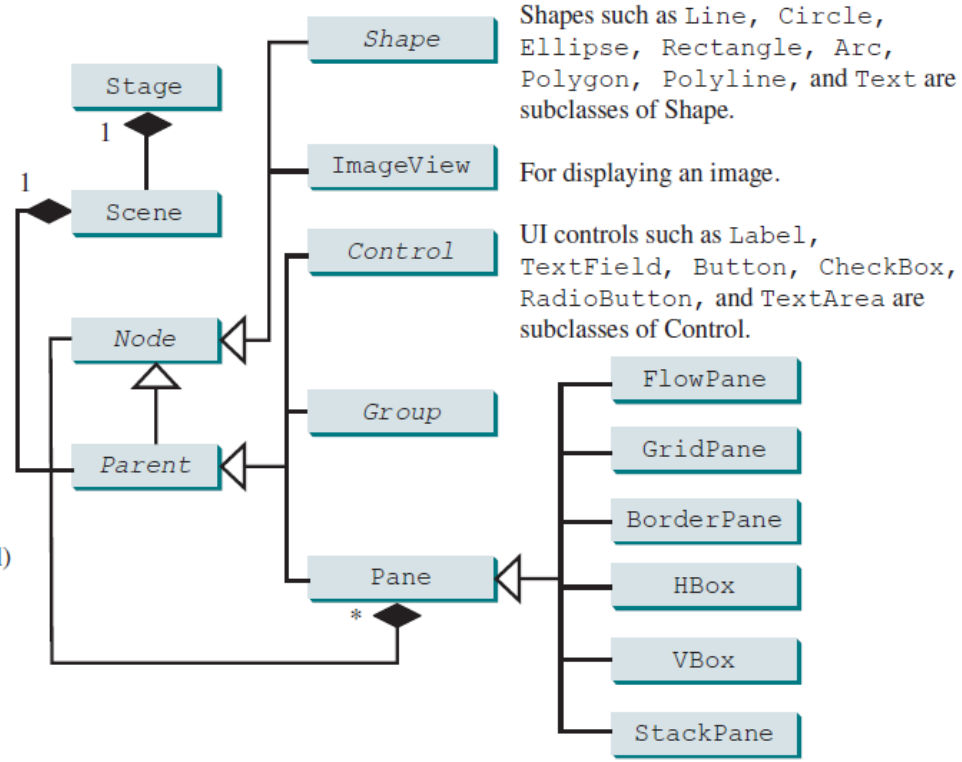


- There are two types of nodes: **branch** nodes and **leaf** nodes
  - a branch node is a node that can contain other nodes (child nodes)
  - branch nodes are also referred to as parent nodes because they can contain child nodes
  - a leaf node is a node which **cannot** contain other nodes

Parent, Panes , Group

- **Parent** component manages the *layout* of the components nested inside it
- We place nodes inside a **Pane** to set the location and the size in a scene
- **Group** object is a collection of nodes





Controls

- **Controls** are JavaFX components which provide UI control functionality
  - for instance, a button, radio button, table, tree
  - for a control to be visible it must be attached to the scene graph of some Scene object
  - controls are usually nested inside some JavaFX layout component that manages the layout of controls relative to each other

- JavaFX contains the following controls:

Accordion	DatePicker	ProgressBar	SplitPane	TitledPane
Button	Label	RadioButton	TableView	ToggleButton
CheckBox	ListView	Slider	TabPane	ToolBar
ChoiceBox	Menu	Spinner	TextArea	TreeTableView
ColorPicker	MenuBar	SplitMenuButton	TextField	TreeView
ComboBox	PasswordField			

JavaFX Application Class

- The primary launch class that has to extend the `Application` class:

```
import javafx.application.Application;



public class JavaFXTesting extends Application {



}
```

Implementing start()

- The start() method is called when JavaFX app is

```
import javafx.application.Application;
import javafx.stage.Stage;



public class JavaFXTesting extends Application {

    @Override
    public void start(Stage primaryStage) {


        primaryStage.setTitle("My First JavaFX Program");

        primaryStage.show();
    }




}
```

start() takes a single parameter of type Stage

set a title on the Stage object

show() is called to make the app visible in a window
if not called, no window is opened

Adding a main() method

- We can launch JavaFX app without main method, but need one to pass parameters

```
import javafx.application.Application;
import javafx.stage.Stage;



public class JavaFXTesting extends Application {

    @Override
    public void start(Stage primaryStage) {


        primaryStage.setTitle("My First JavaFX Program");

        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```

but still, it's best to make it explicit which code launch the app

launch() will detect from which class it is called, so we don't have to tell it explicitly what class to launch

Adding a Scene

- Add a Scene object to the Stage along with a simple Button:

```
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;

public class JavaFXTesting extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button helloButton = new Button("Hello World!");
        Scene scene = new Scene(helloButton, 200, 250);
        primaryStage.setTitle("My First JavaFX Program");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```

first, a Button object is created

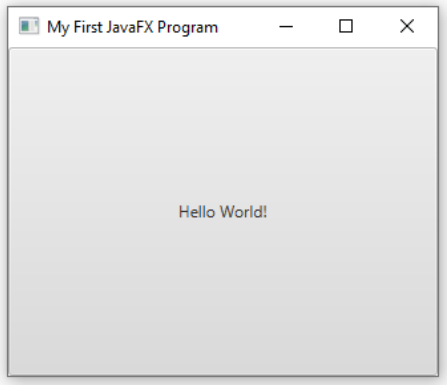
then, a Scene object is created, passing the Button as parameter, along with two parameters representing the width and height of the Scene

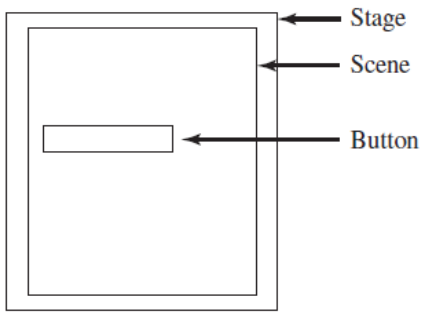
finally, the scene is added to the stage

Hello GUI World!

- Our first JavaFX Program:

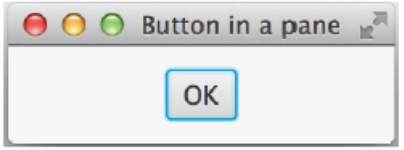
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;

public class JavaFXTesting extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button helloButton = new Button("Hello World!");
        Scene scene = new Scene(helloButton, 200, 250);
        primaryStage.setTitle("My First JavaFX Program");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```





Button in a Pane

- A button is placed in the center of the pane



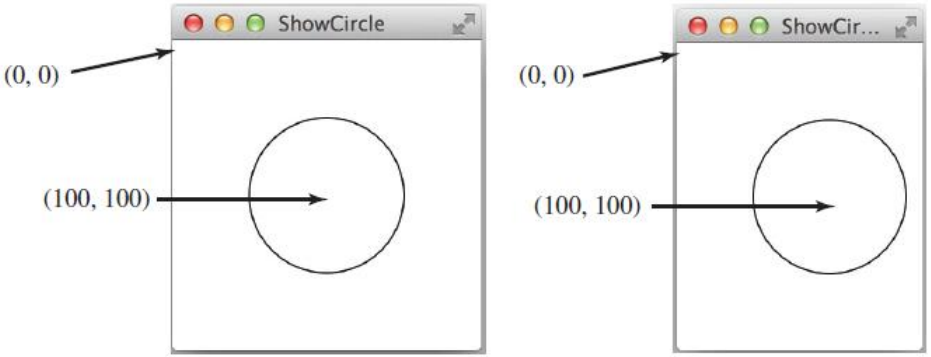

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;

public class ButtonInPane extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place a button in the scene
    StackPane pane = new StackPane();
    pane.getChildren().add(new Button("OK"));
    Scene scene = new Scene(pane, 200, 50);
    primaryStage.setTitle("Button in a pane"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
}
```

```java
StackPane pane = new StackPane(new Button("OK"));
```

Circle

- **Circle** is displayed in the center of scene, but not after window is resized

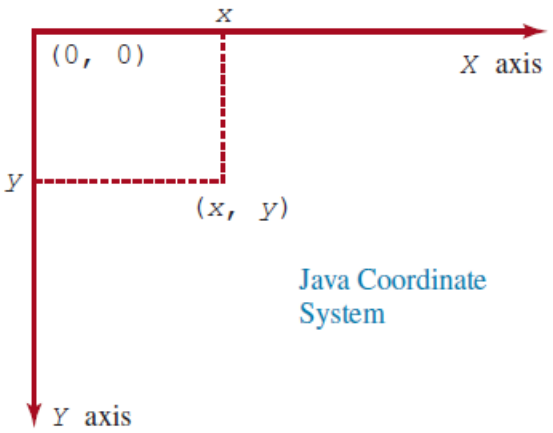
```
// Create a circle and set its properties
Circle circle = new Circle();
circle.setCenterX(100);
circle.setCenterY(100);
circle.setRadius(50);
circle.setStroke(Color.BLACK);
circle.setFill(Color.WHITE);

// Create a pane to hold the circle
Pane pane = new Pane();
pane.getChildren().add(circle);

// Create a scene and place it in the stage
Scene scene = new Scene(pane, 200, 200);
primaryStage.setTitle("ShowCircle"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

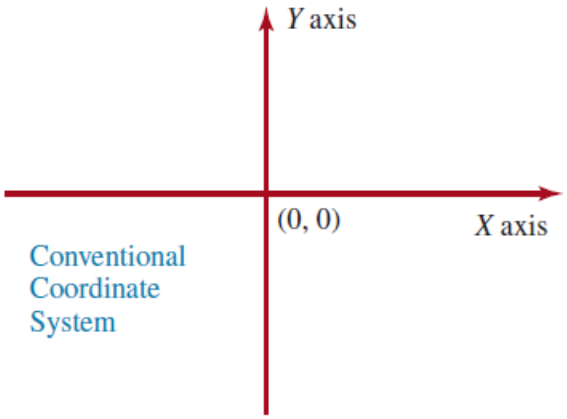


*please check **main reading** for complete code, we only display code/partial code inside **start** method*

Java Coordinate System

- Java coordinate system is measured in pixels, with (0, 0) at its upper-left corner





Property Binding to Center Circle on Resizing

- Property binding enables *a target object* to be bound to *a source object*
  - to automatically change centerX, centerY, **bind** them with width/2, height/2

```
// Create a pane to hold the circle
Pane pane = new Pane();

// Create a circle and set its properties
Circle circle = new Circle();
circle.centerXProperty().bind(pane.widthProperty().divide(2));
circle.centerYProperty().bind(pane.heightProperty().divide(2));
circle.setRadius(50);
circle.setStroke(Color.BLACK);
circle.setFill(Color.WHITE);
pane.getChildren().add(circle); // Add circle to the pane

// Create a scene and place it in the stage
Scene scene = new Scene(pane, 200, 200);
primaryStage.setTitle("ShowCircleCentered"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

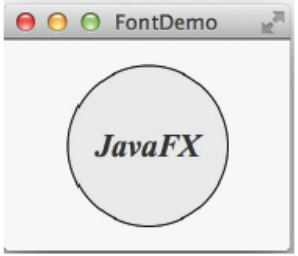
Color

- **Color** class is used to create colors
- Use **RGBA** (red, green, blue, alpha/opacity) model
  - values are from **0.0** (darkest) to **1.0** (lightest)
  - example: **Color** color = new **Color**(0.25, 0.14, 0.333, 0.51);
- Alternatively, use constants for standard colors
  - BEIGE, BLACK, BLUE, BROWN, CYAN, DARKGRAY, GOLD, GRAY, GREEN, LIGHTGRAY, MAGENTA, NAVY, ORANGE, PINK, RED, SILVER, WHITE, and YELLOW
  - example: circle.setFill(**Color.RED**);

Font, Label and StackPane

- **Font** is defined by its name, weight, posture, and size; and is set on a **Label** object

```
// Create a pane to hold the circle
Pane pane = new StackPane();

// Create a circle and set its properties
Circle circle = new Circle();
circle.setRadius(50);
circle.setStroke(Color.BLACK);
circle.setFill(new Color(0.5, 0.5, 0.5, 0.1));
pane.getChildren().add(circle); // Add circle to the pane

// Create a label and set its properties
Label label = new Label("JavaFX");
label.setFont(Font.font("Times New Roman",
  FontWeight.BOLD, FontPosture.ITALIC, 20));
pane.getChildren().add(label);

// Create a scene and place it in the stage
Scene scene = new Scene(pane);
primaryStage.setTitle("FontDemo"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```



- A **StackPane** object is used to contain a circle and a label

Image and ImageView

- **Image** class loads an image; **ImageView** is a node for displaying image

```
// Create a pane to hold the image views
Pane pane = new HBox(10);
pane.setPadding(new Insets(5, 5, 5, 5));
Image image = new Image("image/us.gif");
pane.getChildren().add(new ImageView(image));

ImageView imageView2 = new ImageView(image);
imageView2.setFitHeight(100);
imageView2.setFitWidth(100);
pane.getChildren().add(imageView2);

ImageView imageView3 = new ImageView(image);
imageView3.setRotate(90);
pane.getChildren().add(imageView3);

// Create a scene and place it in the stage
Scene scene = new Scene(pane);
primaryStage.setTitle("ShowImage"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

- An **HBox** is a pane that places all nodes horizontally in one row

Layout Panes

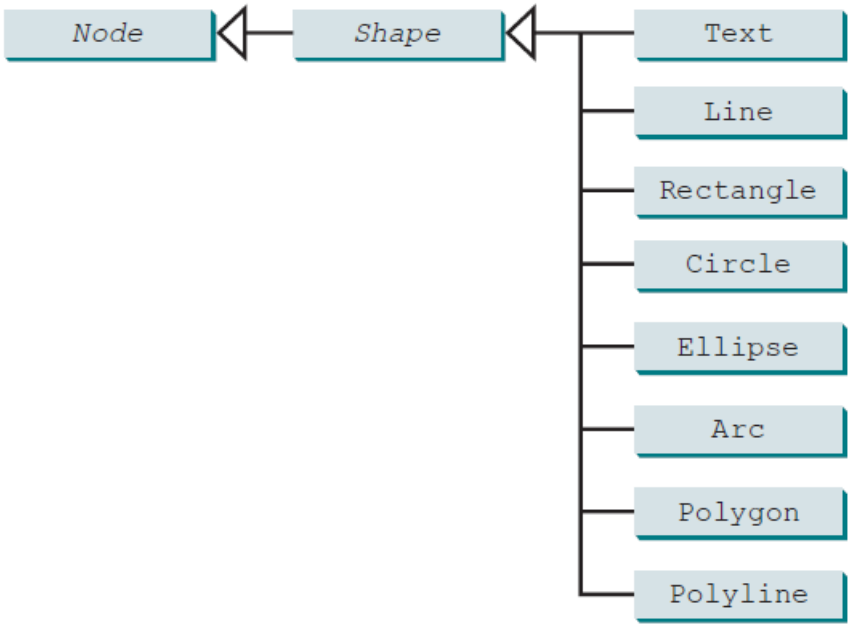
- JavaFX provides many types of panes for organizing nodes

Class	*Description*
`Pane`	Base class for layout panes. It contains the `getChildren()` method for returning a list of nodes in the pane.
`StackPane`	Places the nodes on top of each other in the center of the pane.
`FlowPane`	Places the nodes row-by-row horizontally or column-by-column vertically.
`GridPane`	Places the nodes in the cells in a two-dimensional grid.
`BorderPane`	Places the nodes in the top, right, bottom, left, and center regions.
`HBox`	Places the nodes in a single row.
`VBox`	Places the nodes in a single column.

Shapes

- Shapes are nodes used for drawing texts, lines, circles, rectangles, ellipses, arcs, polygons, and polylines; with properties:

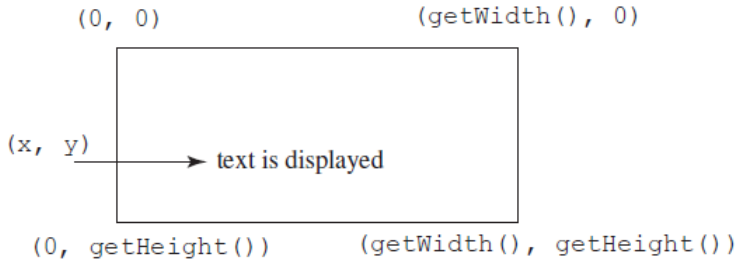
  - **fill**: a color fills the interior of a shape
  - **stroke**: a color used to draw the outline of a shape
  - **strokeWidth**: width of outline of a shape



Text

- **Text** displays a string at a starting point (x, y), placed in a pane

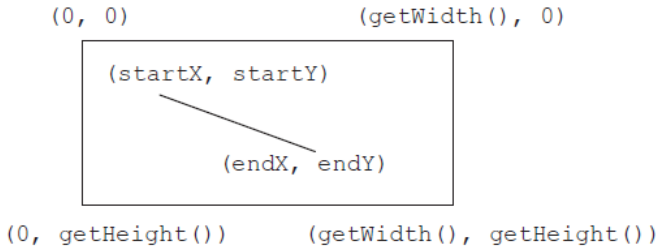
```
// Create a pane to hold the texts
Pane pane = new Pane();
pane.setPadding(new Insets(5, 5, 5, 5));
Text text1 = new Text(20, 20, "Programming is fun");
text1.setFont(Font.font("Courier", FontWeight.BOLD,
  FontPosture.ITALIC, 15));
pane.getChildren().add(text1);

Text text2 = new Text(60, 60, "Programming is fun\nDisplay text");
pane.getChildren().add(text2);

Text text3 = new Text(10, 100, "Programming is fun\nDisplay text");
text3.setFill(Color.RED);
text3.setUnderline(true);
text3.setStrikethrough(true);
pane.getChildren().add(text3);

// Create a scene and place it in the stage
Scene scene = new Scene(pane);
primaryStage.setTitle("ShowText"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

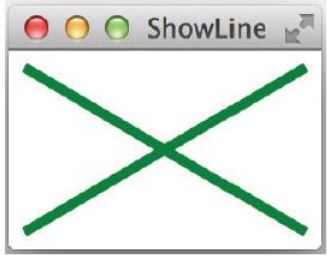


Line

- **Line** connects two points with four parameters **startX**, **startY**, **endX**, and **endY**

```java
public class ShowLine extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place it in the stage
    Scene scene = new Scene(new LinePane(), 200, 200);
    primaryStage.setTitle("ShowLine"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
}
class LinePane extends Pane {
  public LinePane() {
    Line line1 = new Line(10, 10, 10, 10);
    line1.endXProperty().bind(widthProperty().subtract(10));
    line1.endYProperty().bind(heightProperty().subtract(10));
    line1.setStrokeWidth(5);
    line1.setStroke(Color.GREEN);
    getChildren().add(line1);

    Line line2 = new Line(10, 10, 10, 10);
    line2.startXProperty().bind(widthProperty().subtract(10));
    line2.endYProperty().bind(heightProperty().subtract(10));
    line2.setStrokeWidth(5);
    line2.setStroke(Color.GREEN);
    getChildren().add(line2);
  }
}
```



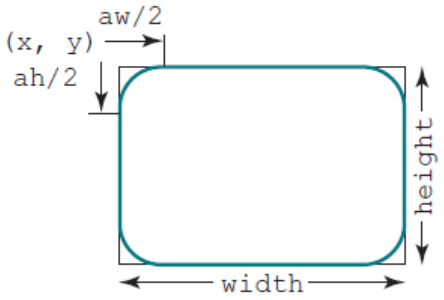


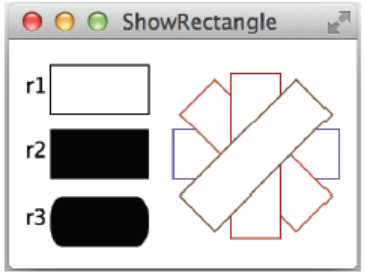
- A custom pane class, **LinePane**, creates two lines and binds the ending points with width, height of pane so two lines are changed when pane is resized

Rectangle

- **Rectangle** is defined by **x**, **y**, **width**, **height**, **arcWidth**, and **arcHeight**

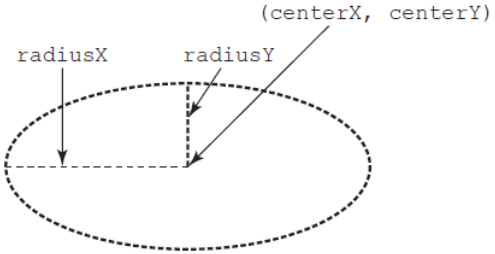
```
// Create rectangles
Rectangle r1 = new Rectangle(25, 10, 60, 30);
r1.setStroke(Color.BLACK);
r1.setFill(Color.WHITE);
Rectangle r2 = new Rectangle(25, 50, 60, 30);
Rectangle r3 = new Rectangle(25, 90, 60, 30);
r3.setArcWidth(15);
r3.setArcHeight(25);
// Create a group and add nodes to the group
Group group = new Group();
group.getChildren().addAll(new Text(10, 27, "r1"), r1,
  new Text(10, 67, "r2"), r2, new Text(10, 107, "r3"), r3);

for (int i = 0; i < 4; i++) {
  Rectangle r = new Rectangle(100, 50, 100, 30);
  r.setRotate(i * 360 / 8);
  r.setStroke(Color.color(Math.random(), Math.random(),
    Math.random()));
  r.setFill(Color.WHITE);
  group.getChildren().add(r);
}
// Create a scene and place it in the stage
Scene scene = new Scene(new BorderPane(group),250, 150);
```



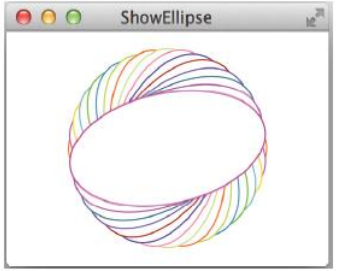


Ellipse

- **Ellipse** is defined by **centerX**, **centerY**, **radiusX**, and **radiusY**




```java
// Create a scene and place it in the stage
Scene scene = new Scene(new MyEllipse(), 300, 200);

class MyEllipse extends Pane {
  private void paint() {
    getChildren().clear();
    for (int i = 0; i < 16; i++) {
      // Create an ellipse and add it to pane
      Ellipse e1 = new Ellipse(getWidth() / 2, getHeight() / 2,
        getWidth() / 2 - 50, getHeight() / 2 - 50);
      e1.setStroke(Color.color(Math.random(), Math.random(),
        Math.random()));
      e1.setFill(Color.WHITE);
      e1.setRotate(i * 180 / 16);
      getChildren().add(e1);
    }
  }

  @Override
  public void setWidth(double width) {
    super.setWidth(width);
    paint();
  }

  @Override
  public void setHeight(double height) {
    super.setHeight(height);
    paint();
  }
```

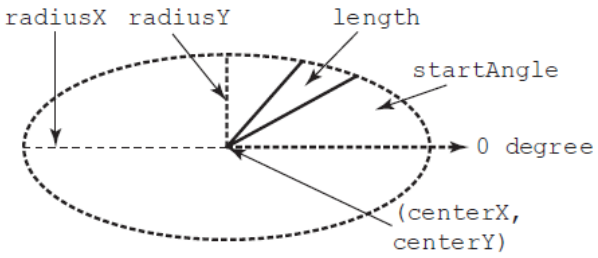


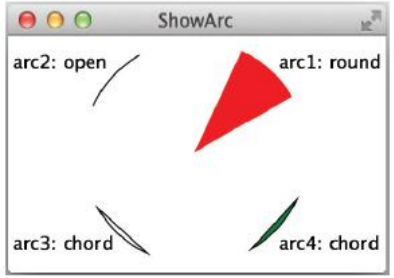
- By defining **MyEllipse** class, we can reuse it, and since it can be resized since it extends Pane and overrides setWidth and setHeight methods

Arc

- **Arc**, part of an ellipse, defined by **centerX**, **centerY**, **radiusX**, **radiusY**, **startAngle**, **length**, and **an arc type** (ArcType.OPEN, ArcType.CHORD, or ArcType.ROUND)

```
Arc arc1 = new Arc(150, 100, 80, 80, 30, 35); // Create an arc
arc1.setFill(Color.RED); // Set fill color
arc1.setType(ArcType.ROUND); // Set arc type

Arc arc2 = new Arc(150, 100, 80, 80, 30 + 90, 35);
arc2.setFill(Color.WHITE);
arc2.setType(ArcType.OPEN);
arc2.setStroke(Color.BLACK);

Arc arc3 = new Arc(150, 100, 80, 80, 30 + 180, 35);
arc3.setFill(Color.WHITE);
arc3.setType(ArcType.CHORD);
arc3.setStroke(Color.BLACK);

Arc arc4 = new Arc(150, 100, 80, 80, 30 + 270, 35);
arc4.setFill(Color.GREEN);
arc4.setType(ArcType.CHORD);
arc4.setStroke(Color.BLACK);
// Create a group and add nodes to the group
Group group = new Group();
group.getChildren().addAll(new Text(210, 40, "arc1: round"),
  arc1, new Text(20, 40, "arc2: open"), arc2,
  new Text(20, 170, "arc3: chord"), arc3,
  new Text(210, 170, "arc4: chord"), arc4);
// Create a scene and place it in the stage
Scene scene = new Scene(new BorderPane(group), 300, 200);
```

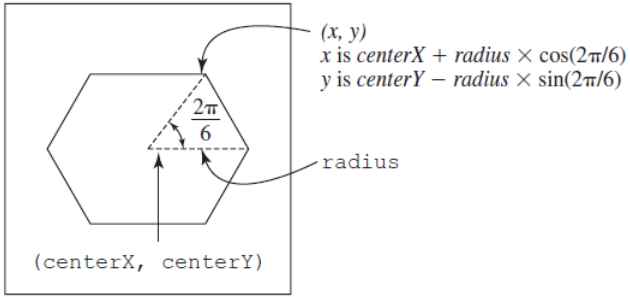


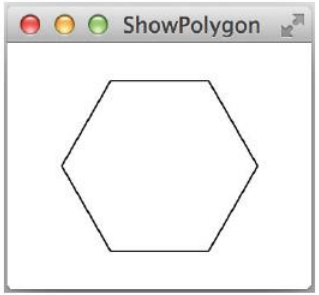


Polygon and Polyline

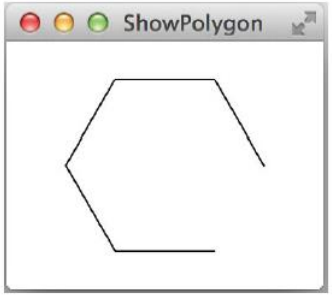
- **Polygon** connects a sequence of points

```
class MyPolygon extends Pane {
  private void paint() {
    // Create a polygon and place polygon to pane
    Polygon polygon = new Polygon();
    polygon.setFill(Color.WHITE);
    polygon.setStroke(Color.BLACK);
    ObservableList<Double> list = polygon.getPoints();

    double centerX = getWidth() / 2, centerY = getHeight() / 2;
    double radius = Math.min(getWidth(), getHeight()) * 0.4;

    // Add points to the polygon list
    for (int i = 0; i < 6; i++) {
      list.add(centerX + radius * Math.cos(2 * i * Math.PI / 6));
      list.add(centerY - radius * Math.sin(2 * i * Math.PI / 6));
    }

    getChildren().clear();
    getChildren().add(polygon);
  }

  @Override
  public void setWidth(double width) {
    super.setWidth(width);
    paint();
  }
  @Override
  public void setHeight(double height) {
    super.setHeight(height);
    paint();
  }
}
```





- If replace Polygon with **Polyline**, similar display, except the starting and ending points are not connected



Optional Material

- The next few slides are for CW3
  - we have received some questions about them

- *Not* tested in the Final Exam

Switch

- The switch statement selects a block of code to run based on a value
  - Evaluates one expression and matches it against case labels
  - Includes an optional default case
  - Example:

```
int day = 3;
switch (day) {
      case 1:
            System.out.println("Monday");
            break;
      case 2:
            System.out.println("Tuesday");
            break;
      default:
            System.out.println("Weekend");
}
```

Enhanced Switch

- Alternatively, we can use Enhanced Switch (Java 14+)

```
int day = 3;
switch(day) {
    case 1 -> System.out.println("Monday");
    case 2 -> System.out.println("Tuesday");
    case 3 -> System.out.println("Wednesday");
    default -> System.out.println("Invalid day");
}
```

Switch with Strings

- Java supports switching on Strings (since Java 7)

```
String role = "admin";
switch (role) {
    case "admin" -> System.out.println("Full access");
    case "editor" -> System.out.println("Edit access");
    case "viewer" -> System.out.println("Read-only access");
    default -> System.out.println("No access");
}
```

Charts

- JavaFX comes with a set of built-in ready-to-use **chart** components, so you don't have to code charts from scratch every time you need a basic chart

- JavaFX contains the following chart components:
  - AreaChart
  - BarChart
  - BubbleChart
  - LineChart
  - PieChart
  - ScatterChart
  - StackedAreaChart
  - StackedBarChart

Events

- In JavaFX, we can develop GUI applications, user interacts with the application (nodes), an **event** is said to have been occurred
  - for example, clicking on a button, moving the mouse, entering a character through keyboard, selecting an item from list
- An **Event** is any change of state in an input device (such as mouse or touch screen), a user action (ActionEvent), or a background task
- There are **90 separate types** of event supported by JavaFX, with the capability to extend the Event class and define additional custom functionality

- JavaFX then provides significant support to run executable code, defined separately through the **EventHandler** class, on any changes of state

EventHandler

- The **EventHandler** object is an object with a single method, which we need to override when we instantiate it
- As the EventHandler interface is parameterized, we need to specify what event type we are handling
  - for example, MouseEvent

```
EventHandler<MouseEvent> eventHandler = new EventHandler<>() {
    @Override
    public void handle(MouseEvent event) {
        //executable code
    }
};
```

Event Handling: Anonymous Class vs Lambda Expression

- For CW3, if you use Event Handler, please use **Anonymous Class**:

```
//Creating the event handler
EventHandler<Event> eventHandler = new EventHandler<Event>() {
    @Override
    public void handle(Event e) {
        //do stuff
    }
};
```

- Just for your information (*don't* use for CW3, *not* in syllabus), for convenience, the handle() method can be specified with *a lambda expression*:

```
//Creating the event handler
EventHandler<Event> eventHandler = (event) -> {
    //do stuff
};
```

Thank you for your attention !

- In this lecture, you have learned:
  - how develop rich GUI applications with JavaFX, using a stage, scenes and nodes
  - how to draw shapes such as circles, lines, rectangles, ellipses, arcs, polygons, and polylines

- Please continue to Lab 12 to complete Lab Tasks